

Building a Course Registration System

How I turned a stressful student experience into working software

Your Full Name • Student ID • English Writing & Presentation Skills

Lecturer's Name • October 5, 2026

1. Introduction

At the start of every semester, thousands of students at my university sit in front of their computers and wait for the course registration page to open. When it finally does, everyone clicks at the same time. Within minutes, popular classes are full, some students discover that two of their classes are at the same time, and many are not even sure whether their registration was saved. I have been through this experience several times, and it was always stressful.

This experience inspired the project that I would like to describe in this report: a **Course Management and Registration System**. It is a web application that I designed and built by myself between January and March 2026. Students use it to register for classes and see their timetable and grades, while the academic office uses it to manage courses, semesters and classes.

In this report, I will first explain why I chose this project and what I wanted to achieve. Next, I will describe what the system does and how I built it. After that, I will focus on the two biggest challenges I faced and how I solved them. Finally, I will reflect on what I learned from the experience.

2. Why I Chose This Project

I chose this project for two main reasons. The first reason is personal: as a student, I understand the problem very well, so I knew exactly what a good system should do. The second reason is professional: I wanted a project that was large enough to practise everything I had learned in my software courses, from designing a database to building a user interface.

Before writing any code, I set myself four clear goals:

1. to build a **complete** system with two separate areas, one for students and one for administrators;
2. to make sure the system is **fair and reliable**, so that it never accepts more students than a class can hold;
3. to make it **easy to use**, with clear messages when something goes wrong;

4. to make it **easy to install**, so that anyone can run it with a single command.

3. What the System Does

The system has two “portals”, or areas, and each user only sees the area that matches their role. Table 1 summarises the main features of each one.

Student portal	Administrator portal
View personal information and study status	Create and update courses and their prerequisites
Register for a class, or cancel it, with one click	Set up semesters and the registration period
See a weekly timetable of all registered classes	Open classes and assign lecturers and rooms
Look up scores, letter grades and GPA	Monitor how many students have registered

Table 1: Main features of the two portals

For example, when a student wants to take a class, they simply open the registration page, choose a class and click *Register*. Behind the scenes, the system checks several rules before it accepts the request. If one rule is broken, for instance if the class is already full, the student sees a clear message explaining why. I also added a *dark mode*, which is more comfortable for the eyes when working at night.

4. How I Built It

4.1 The three main parts

Like most modern web applications, my system is made of three parts that work together, as shown in Figure 1.

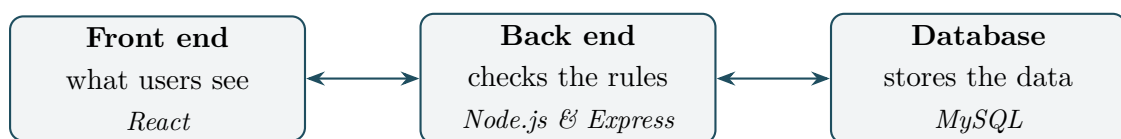


Figure 1: The three main parts of the system

The **front end** is the part that users see and click on in their web browser. I built it with React, a popular tool for creating interactive web pages. The **back end** runs on a server. It receives every request, checks who the user is, applies the registration rules and then sends back an answer. I wrote it in JavaScript using Node.js and Express. Finally, the **database** stores all the information, such as students, courses, classes, timetables and grades, in eight connected tables. I used MySQL for this part.

To keep the system secure, every user must log in, and passwords are never stored as plain

text. In addition, I packaged all three parts with a tool called *Docker*, so that the whole system can be started with a single command on any computer.

4.2 My working process

I organised the work into five stages:

1. **Analysis (January).** I studied how registration works at my university and wrote down nine rules that the system must follow, for example “a student cannot register for a full class”.
2. **Database and back end (March).** I designed the database and built the back end, which contains all the rules.
3. **Reorganisation.** As the project grew, I restructured the code so that the front end and the back end live in one well-organised project.
4. **Front end.** I built the student and administrator websites and improved the design.
5. **Deployment and documentation.** I prepared the system for installation with Docker and wrote installation and technical documentation.

Throughout the project, I used Git, a version control tool, to save every step of my work. This meant that I could always go back if I made a mistake.

5. The Biggest Challenges

Although many parts of the project were straightforward, two problems took much more time and thought than I had expected.

5.1 Challenge 1: the last seat

Imagine a class with 40 seats, and 39 students have already registered. Now, two students click *Register* at exactly the same moment. In a simple system, both requests ask the database “Is there a free seat?”, both receive the answer “Yes, one seat”, and both students are accepted. As a result, the class ends up with 41 students, which is clearly unfair and causes problems for the lecturer.

At first, I did not even notice this problem, because it never happened when I tested the system alone. However, after reading about how real registration systems work, I realised that it would definitely happen on a busy registration day. To solve it, I used a database feature called a **transaction with a lock**. In simple terms, when the first student starts registering for a class, the system “locks” that class for a fraction of a second. The second student has to wait until the first registration is complete. When it is their turn, the system correctly sees that the class is full and shows a polite message. In addition, I added a rule inside the database itself, so that the number of students can never be higher than the number of seats, even if there were a mistake in my code.

5.2 Challenge 2: timetable clashes

The second challenge was to stop students from registering for two classes that take place at the same time. This sounds easy, but real timetables are more complicated than they look. Some classes, for instance, only meet in odd weeks, while others only meet in even weeks. I eventually defined a clear rule: two classes clash only if *all three* of the following are true:

- they are on the same day of the week;
- their lesson periods overlap;
- their weeks are compatible (an odd-week class never clashes with an even-week class).

Whenever a clash is found, the system does not simply say “Error”. Instead, it tells the student exactly which of their classes is causing the problem, so they know what to change.

6. What I Learned

This project taught me much more than any single course exercise. On the technical side, I became confident with all three parts of a web application, and I now understand why reliability problems like “the last seat” matter so much in real systems. More importantly, however, I developed several skills that I believe will be useful in any job.

- **Planning before acting.** Writing down the nine rules before coding saved me a great deal of time, because I always knew exactly what I was building.
- **Learning independently.** Many solutions, such as the transaction lock, were not taught in my classes. I had to find information in documentation and articles, most of which were written in English.
- **Writing clearly.** I wrote documentation so that other people could install and use the system. This taught me to explain technical ideas in simple, well-organised language, which is also the aim of this report.
- **Managing my time.** Breaking the work into stages helped me to make steady progress and to stay motivated.

If I had more time, I would like to add automatic tests, a waiting list for full classes with email notifications, and an automatic check that students have completed the required prerequisite courses.

7. Conclusion

In conclusion, the Course Management and Registration System is the project I am most proud of so far. It started from a problem that I experienced myself as a student, and it became a complete system that is fair, easy to use and easy to install. Along the way, I faced real challenges, such as protecting the last seat in a class and detecting timetable clashes,

and solving them taught me to plan carefully, learn independently and communicate clearly. Above all, this project showed me that good software is not only about writing code, but about understanding people's problems and solving them in a reliable way.